

ml-lab-ass-4

July 18, 2025

1 Assignment 4

1.1 Step 1: Import Required Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, \
    recall_score
```

1.2 Step 2: Load the Dataset

```
[2]: # Load the dataset
df = pd.read_csv('diabetes.csv') # Make sure the CSV is in the same folder as \
    your notebook
df.head()
```

```
[2]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	Pedigree	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

1.3 Step 3: Data Preprocessing

```
[3]: # Check for missing values
print(df.isnull().sum())

# Statistical overview
print(df.describe())

# Feature Scaling
X = df.drop('Outcome', axis=1)
y = df['Outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
Pregnancies      0
Glucose           0
BloodPressure     0
SkinThickness     0
Insulin           0
BMI               0
Pedigree          0
Age              0
Outcome           0
```

```
dtype: int64
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479
std	3.369578	31.972618	19.355807	15.952218	115.244002
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

	BMI	Pedigree	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

1.4 Step 4: Train-Test Split

```
[4]: X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
    ↪random_state=42)
print(f'Training size: {len(X_train)}, Testing size: {len(X_test)}')
```

Training size: 614, Testing size: 154

1.5 Step 5: Apply K-Nearest Neighbors (KNN) Algorithm

```
[5]: # Create and train the model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# Predict on test set
y_pred = knn.predict(X_test)
```

1.6 Step 6: Confusion Matrix & Evaluation Metrics

```
[6]: # Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

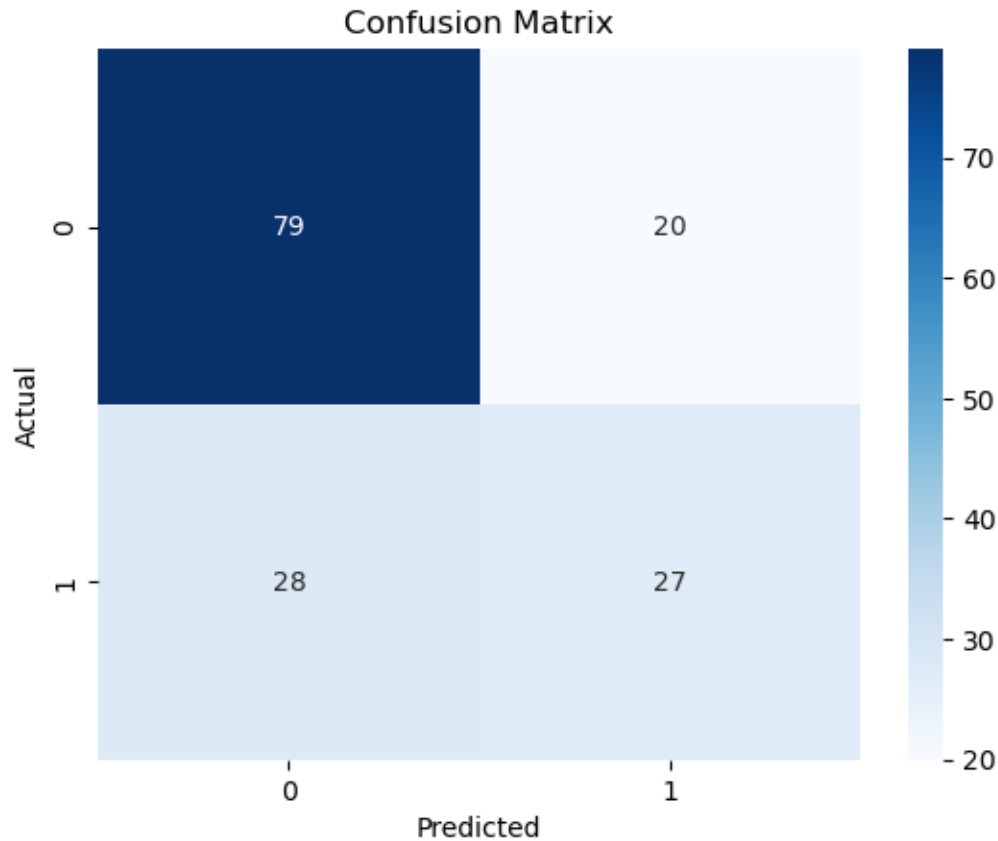
# Plotting Confusion Matrix
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# Metrics
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
```

Confusion Matrix:

```
[[79 20]
 [28 27]]
```



Accuracy: 0.69
Error Rate: 0.31
Precision: 0.57
Recall: 0.49

1.7 Observation

1. The model performs reasonably well at predicting non-diabetic cases ($TN = 79$), but It struggles more with detecting diabetic patients (only $TP = 27$, $FN = 28$).
2. Recall (0.49) is low, indicating the model misses many actual diabetic patients. This could be risky in real-world applications, where catching all true positive cases (diabetes) is critical.
3. The precision (0.57) is slightly better, meaning when it says someone has diabetes, it's correct 57% of the time. ## Conclusion Overall, the model leans toward predicting “no diabetes” more often, possibly due to class imbalance (more 0s than 1s in the dataset).